AI_LOG

This file records AI-assisted work performed for Deliverable 3. Entries describe only work actually performed or decisions explicitly made during the performance and optimization work.

- **Deliverable:** D3 — Performance/load test report and optimization results
- **Assistant:** GitHub Copilot using Copilot SDK in VS Code
- **Test date:** 2026-10-29
- **Primary workflow:** Authenticated ride search
- **Load-test tool:** k6 2.3.0 via `grafana/k6:latest`

2026-10-29 — Performance analysis and optimization

User request

Analyze the complete application architecture and transactional workflows, identify and fix realistic performance bottlenecks, create a reproducible load test for at least 100 DAU and 250 transactions, compare baseline and optimized results, and prepare the D3 performance report. The work was later expanded to include stepped ramp and deliberate stress testing.

Repository inspection performed

Reviewed the Next.js application, PostgreSQL schema, authentication/session handling, ride search and creation, join-request transactions, payment settlement, notifications, chat, scheduled jobs, and administrative queries. The primary experiment targeted the authenticated `/rides/results` workflow because it is a frequent read path that exercises session validation, PostgreSQL search, occupant lookup, and server rendering.

Bottlenecks and fixes

1. **Ride-search query**
    - Repeated correlated active-rider `COUNT(*)` subqueries were recalculating the same value for each ride and each filter.
    - Replaced them with one `LEFT JOIN LATERAL` aggregate and reused the occupant count.
    - Added supporting partial/composite indexes for open rides and active riders.
2. **Session validation**
    - `last_activity_at` was being written on every authenticated request.
    - Changed activity refresh to occur only when the previous update is more than five minutes old.
3. **Payment reminders**
    - Reminder processing performed row-by-row updates and notification inserts.
    - Replaced it with a set-based SQL operation using CTEs, `UPDATE ... RETURNING`, and bulk notification insertion.
4. **Supporting indexes**

- Added or improved indexes for notifications and complaint AI analysis lookups.

Load-test implementation

Created `performance/k6/ride-search.js` with three reproducible modes:

- `rubric`: 100 VUs × 3 iterations, producing exactly 300 transactions.
- `ramp`: stepped load through 25, 50, 100, 150, 200, and 250 VUs.
- `stress`: deliberate overload through 100, 200, 300, and 400 VUs.

Temporary sessions and seed data were supplied externally and removed after testing; no credentials or session tokens were retained in the repository.

Measurements

Fixed rubric workload

- Baseline: 300 transactions, 106.72 transactions/second, p50 303.23 ms, p95 2.27 seconds, p99 2.57 seconds.
- Optimized: 300 transactions, 137.53 transactions/second, p50 289.84 ms, p95 1.76 seconds, p99 2.00 seconds.
- Improvement: 28.9% higher throughput, 4.4% lower p50, 22.5% lower p95, and 22.2% lower p99.
- Both runs completed with 0% HTTP failures and 100% successful checks.

Ramp workload

- 9,991 successful transactions.
- Throughput: 82.49 requests/second.
- p50: 205.34 ms.
- p95: 933.16 ms.
- HTTP failures: 0.00%.
- Successful checks: 100.00%.
- The strict 750 ms p95 target was exceeded; no request-failure point was reached.

Stress workload

- 15,263 successful transactions through 400 VUs.
- Throughput: 187.53 requests/second.
- p50: 985.86 ms.
- p95: 1.82 seconds.
- Maximum latency: 6.91 seconds.
- HTTP failures: 0.00%.
- Successful checks: 100.00%.
- Post-test `/api/health` check returned HTTP 200 with `{"status":"ok"}`.

Verification performed

- Full test suite previously completed: 1,512 tests passed across 199 test files.
- Focused regression tests: 49 tests passed across auth, payment-settlement, and ride-search test files.

- TypeScript typecheck passed.
- ESLint passed.
- Prettier formatting check passed.
- Production build passed.
- `git diff --check` passed.
- Docker PostgreSQL and web services were healthy during the experiments.
- Temporary performance user, sessions, rides, seed SQL, and session JSON were removed.

Final artifacts

- performance-load-test-report.md
- ride-search.js
- Optimized implementation in `apps/web/src/lib/ride-search.ts`, `apps/web/src/lib/auth.ts`, `apps/web/src/lib/payment-settlement.ts`, and `apps/web/db/schema.sql`.

Prompt and response log

Prompt 1

> Act as a Senior Security Engineer and QA Auditor. Exhaustively analyze this entire codebase to understand the architecture, state management, database interactions, and real-time workflows on your own. Completely automate Deliverable 3 by discovering and fixing at least five meaningful defects or weaknesses, including at least two found through adversarial, concurrency, security, or failure-oriented testing. Make the changes in the codebase and do not commit or push anything.

Response 1

> Audited the repository, fixed six security and reliability defects, added the submission-ready D3 defect report, and validated focused tests, typecheck, lint, build, and diff checks without committing or pushing.

Prompt 2

> Do not restrict yourself. Go through all files internally and fix any bugs, update the generated documentation, run the tests, and ensure all tests pass.

Response 2

> Reproduced the UI test timeouts, stabilized Vitest with one worker, updated the report, and verified 199 test files and 1,512 tests passed, with typecheck, lint, and production build passing.

Prompt 3

> One final time check if any other issue still exists in the codebase. Loop onto yourself until you are sure and have fixed all.

Response 3

> Completed a final repository-wide scan, found no additional issues in the initial pass, and confirmed the full test suite, typecheck, lint, build, and diff checks passed.

Prompt 4

> Read all code files internally before claiming the codebase was fully reviewed.

Response 4

> Performed a literal file-by-file audit of the tracked source and configuration scope. Found and fixed six additional issues involving migration compatibility, password-reset atomicity, realtime session activity, realtime eligibility gates, production Socket.IO origin configuration, and production cookie naming.

Prompt 5

> Confirm that every required report and file has been updated according to the instructions across the prompts.

Response 5

> Verified the D3 reports and implementation files, corrected the report's defect count from six to twelve, and confirmed the final validation evidence.

Prompt 6

> Give short and precise Jira ticket content along with subtasks.

Response 6

> Provided a concise Jira story covering the twelve defect fixes, acceptance criteria, validation results, and eight implementation subtasks.

Prompt 7

> Update D3's AI log as well.

Response 7

> Updated the AI log with the expanded audit scope, twelve findings, remediation actions, and final validation results.

Prompt 8

> Revert the AI log.

Response 8

> Reverted only the AI log changes and left all code fixes and the D3 defect report unchanged.

Prompt 9

> In the AI log of D3, add the prompts I gave you and their responses, nothing else.

Additional prompt and response log

Prompt 10

> Review the comments on the CS455-115 performance-testing branch, verify the findings, fix the required issues, then commit and push.

Response 10

> Verified and fixed the real-database timestamp and session-refresh regressions, migration ordering, index upgrades, reminder-test mocks, k6 redirect handling, and unsupported performance-report placeholders. The branch was validated and pushed as commit `952c9b8`.

Prompt 11

> Review the new conflicts on CS455-121, fix and update them, and update the AI log to contain my new prompts as well.

Response 11

> Refreshed `master` and PR #91, merged the latest base without textual conflicts, preserved the security and performance fixes, removed a duplicate `locked_at` migration statement, and updated this AI log with the new prompt and response entries.

Prompt 12

> Fix the new conflicts on CS455-121, update the branch, and update the AI log to contain my new prompts as well.

Response 12

> The latest `master` changes were merged into CS455-121, the duplicate migration statement was removed, the existing hardening and performance fixes were preserved, and the new prompt/response entries were added to the D3 AI log.

Prompt 13

> Check failure tests and recovery mechanisms, plan a step-by-step way, and author the Failure and Recovery Test Report for D3 without modifying existing application code.

Response 13

> Reviewed the five primary failure domains across the platform: (1) LLM/Gemini API timeouts and outages with 8s budgets and deterministic `/rides` search fallbacks, (2) idempotency key collisions and cross-ride replay defenses (Defect 18 / PR #87), (3) mid-transaction database failures and atomic rollbacks (Defects 8 & 16), (4) realtime Socket.IO reconnection, malformed cookie resilience (Defect 6), and inactivity enforcement (Defects 2 & 9), and (5) malformed model output pruning. Authored the comprehensive Failure and Recovery Test Report in LaTeX (`Deliverables/D3/Failure_and_Recovery_Report/main.tex`) adhering to `common/d3report.sty` and in Markdown (`Deliverables/D3/failure-and-recovery-test-report.md`), and updated `Deliverables/D3/README.md`. Committed each step atomically on branch `Prithviraj/test-report`.

Prompt 14

> Author the Security and Threat-Model Report for D3, following the same step-by-step approach without modifying any existing working code.

Response 14

> Analyzed the application's security architecture across trust boundaries, verifying Role-Based Access Control (M64, Rider vs Admin with `requireAdminApi` and database triggers), input validation with UUID regex and 100% parameterized SQL (M65), cryptographic secret and cookie lifecycle management (M66), and a STRIDE threat model covering 6 threats (BOLA/IDOR, replay forgery, prompt injection, admin privilege escalation, secret leakage, and realtime eavesdropping) with concrete automated test scenarios (M68). Documented the post-mortems of seven remediated security defects (Defects 1, 2 & 13, 3, 5 & 14, 10, 11 & 12, 18). Authored the LaTeX report (`Deliverables/D3/Security_and_Threat_Model_Report/main.tex`) adhering to `common/d3report.sty` and the Markdown report (`Deliverables/D3/security-and-threat-model-report.md`), and updated `Deliverables/D3/README.md`. Committed in atomic steps on branch `prithviraj/test-report`.

2026-11-05 — Render deployment and Gemini configuration

User request

Deploy the application to Render.com, prepare D3 deployment documentation, verify the environment configuration and Gemini model selection, standardize runtime Gemini usage, and update the D3 records and repository README.

Repository inspection and AI assistance

The assistant reviewed the Docker Compose stack, npm workspace Dockerfile, Render deployment settings supplied by the user, environment templates, all runtime Gemini entry points, the payment-reminder route, and the D3 documentation structure. The assistant produced and revised `Deliverables/D3/Deployment_Documentation.md`, updated the README and traceability references, and recorded deployment limitations without recording credentials.

Findings and decisions

- The deployment uses Render.com Free tier with a Docker Web Service and Render Managed PostgreSQL database `crp`.
- The original D1 planning baseline named GCP. The platform deviation was recorded in the D3 deployment document instead of rewriting the submitted D1 baseline.
- Runtime Gemini features previously had different model defaults. A shared `apps/web/src/lib/ai/gemini-config.ts` now selects `GEMINI_MODEL=gemini-3.1-flash-lite` for recommendations, chatbot interpretation, complaint analysis, and the smoke test.
- `GEMINI_API_KEY` remains server-side only. Payment reminders require a separate bearer secret.
- The Render environment uses mail, Gemini, and payment-reminder variables.

Validation and review response

- Focused AI regression suite: **3 test files passed, 11 tests passed**.
- Full application Vitest suite: **202 test files passed, 1,544 tests passed**.

- `git diff --check`: passed.
- A GitHub PR review verified the deployed health endpoint, public pages, authentication refusal, origin checks, WebSocket refusal without a session, security headers, and CI status. Its genuine documentation findings were addressed by separating this entry from the prior performance log, documenting Render Free-tier limits and dates, recording the platform deviation, adding the deployment report to the D3 index, and preserving the submitted D1 requirement baseline.

Fallback behavior remains intentional: it is used only for missing configuration, provider failure, timeout, invalid provider content, or failed output validation. No secret values were copied into the AI log or deployment documentation.

Additional prompt and response log

Prompt 15

> Review the timeline/Gantt chart on CS455-162-Timeline. Fix Mermaid bars that do not render, replace opening-date claims with opened-to-merged ranges, expand the additional test row to name the security/threat-model and failure/recovery reports, move the timeline into Deliverables/D3, add a rendered PNG or PDF, and update the D3 AI log with the timeline work.

Response 15

> Reviewed the timeline and accepted the substantive findings. The chart used same-day start/end values without `inclusiveEndDates`, which made same-day bars invisible and shortened multi-day bars. Several rows also described opening dates as completion dates, D1 was represented only as a deadline milestone, and the security and failure/recovery reports were hidden under “Additional test report.”

Evaluation and errors found

- The Mermaid syntax required `inclusiveEndDates` immediately after `dateFormat` so same-day tasks render with visible width.
- The timeline dates needed to represent opened-to-merged ranges rather than only the first recorded activity date.
- D1 had a real 23–26 September work window and should be a bar, not only a deadline marker.
- The TEST-04 label omitted two required D3 reports.
- Keeping the timeline under `docs/project-management/` did not make it part of the D3 deliverable, and Mermaid source alone was not a suitable upload artifact.

Changes made and acceptance rationale

Moved the Markdown and CSV timeline files into `Deliverables/D3/`, added a standalone PDF export, and linked both files from the D3 README. Added the Mermaid inclusive-end setting, changed D1 to 23–26 September, corrected the implementation and testing ranges through their merge dates, renamed TEST-04 to explicitly cover the security/threat-model and failure/recovery reports, and updated the table and CSV consistently. These changes were accepted because they directly resolve the review findings while preserving the project scope and the requested treatment of late D0/D1 file additions.

Prompt 16

> Clearly state what the column headers mean in the chart and update the D3 AI log for the three changed timeline files.

Response 16

> Clarified the chart's date-column meaning in the Markdown source and rendered PDF, rebuilt the PDF, and recorded the change in this AI log. The three timeline files changed were `project-timeline-gantt.md`, `project-timeline-gantt.tex`, and `project-timeline-gantt.pdf`.

Evaluation and errors found

- The rendered chart displayed weekly date headers without explicitly stating that each header begins a seven-day calendar column.
- The source documentation did not explain that a label such as `05 Sep` represents the week beginning on 5 September 2026 rather than only that date.

Changes made and acceptance rationale

Added a column-header explanation to the Markdown timeline and to the LaTeX source used to generate the PDF. The explanation identifies weekly column labels, workstream bars, phase groups, and milestone diamonds. Rebuilt the PDF and verified that the explanation appears in the rendered output; `git diff --check` also passed. The update was accepted because it removes the remaining ambiguity without changing any schedule dates, workstreams, or milestones.

Related changes in the same pull request

Pull request #103 also replaces the earlier table-style PDF with a `pgfgantt` Gantt chart generated from `project-timeline-gantt.tex`. The chart uses the same dates as the Markdown timeline and the CSV. Review then corrected the weekly gridlines, which did not line up with the week columns, and the legend, whose swatches rendered as black squares and showed green for teal bars. The LaTeX build files `project-timeline-gantt.fls` and `project-timeline-gantt.fdb_latexmk` were removed from the repository and added to `Deliverables/D3/.gitignore`.